

RE001 — Alpha Testing Completion Audit 1

YunSong Fu
s224260453

Food Remedy Mobile App / API Project

1. Introduction

Food Remedy is a capstone software project under Gopher Industries. The product is related to food health evaluation and recommendations. The main aim of the application is to help users make more informed food choices by checking food product information and providing healthier or more suitable alternatives.

The project includes a mobile frontend and backend/data components. The mobile application is built using React Native and Expo. The app can connect to a backend API or use Firestore as a data source, depending on the environment configuration. This means the product needs both frontend testing and backend/API testing before it can be considered ready for wider testing.

The purpose of this report is to assess the current alpha testing evidence and decide whether the application is ready to enter closed beta testing. Alpha testing normally focuses on internal testing by the development team to find major issues before external users test the product. Closed beta testing usually involves a small group of selected users, so the application should be stable enough to install, run, and use without serious blockers.

2. Scope of the Review

This review focuses on the following areas:

1. Existing testing reports and repository evidence
2. Mobile app setup and Expo start-up process
3. Backend/API connection readiness
4. Firestore database and recommendation mode
5. Main feature readiness
6. Unresolved issues that may block closed beta testing
7. Final recommendation on alpha completion and beta readiness

This report does not claim that every file in the project has been fully tested. Instead, it gives an evidence-based assessment using the available information. Where there is no clear proof that a feature works, the feature is marked as **Not Evidenced** rather than automatically marked as failed.

3. Evidence Reviewed

The following evidence was reviewed for this report:

Evidence Area	Description
Mobile app README/setup instructions	Shows the app uses React Native and Expo and provides setup commands such as <code>npm install</code> and <code>npm start</code> .
Environment configuration instructions	Shows the app uses a <code>.env</code> file with variables such as <code>EXPO_PUBLIC_API_BASE_URL</code> , <code>EXPO_PUBLIC_API_SOURCE</code> , and <code>EXPO_PUBLIC_RECOMMENDATION_SOURCE</code> .
Firestore recommendation mode documentation	Shows that the app can use Firestore for recommendation-related data when configured.
Repository structure	Shows backend/API-related folders and project documents.
Food Remedy scoring model research	Explains that the food health evaluation should consider calories together with other nutrition factors such as sugar, saturated fat, protein, fibre, sodium, and ingredient quality.
Capstone role documentation	Shows the planned role includes API development, testing, documentation, task tracking, and small backend tasks.

4. Feature Testing Classification

The table below classifies the main Food Remedy features based on the available evidence.

Feature /	Evidence Found	Status	Reason
Mobile app project	README instructions show Node.js, Expo CLI, <code>npm install</code> , and <code>npm start</code> steps.	Partially Pass	Setup instructions exist, but there was an npm/Expo cache error during setup, so setup may not be smooth
Expo mobile app	Instructions show the app can be started using Expo and previewed through Expo Go.	Partially Pass	The process is documented, but more evidence is needed that the app successfully runs on real iOS and
Environment variable	<code>.env</code> instructions are provided for backend and Firestore modes.	Passed	The required environment variables are clearly listed.
Backend API connecti	The <code>.env</code> file includes <code>EXPO_PUBLIC_API_BASE_URL</code> , with example URL <code>http://127.0.0.1:8000</code> .	Partially Pass	Backend connection is planned, but real phone testing may fail if <code>127.0.0.1</code> is used because the
Firestore data access	Documentation shows <code>EXPO_PUBLIC_API_SOURCE=firebase</code> can force Firestore–	Partially Pass	Firestore mode is supported, but the database content and indexes must be confirmed.

Recommendation	Documentation and research show recommendation logic can use	Partially	The logic is planned, but more testing evidence is needed to prove
Food health scoring	Research suggests using calories together with sugar, saturated fat, protein, fibre, sodium, ingredient	Partially Pass	The scoring concept is well supported, but implementation evidence needs to be confirmed.
Product database	Firestore documentation mentions product documents and category–	Partially	Product data structure is described, but data completeness and quality
Barcode scanning	No clear testing evidence was found from the reviewed material.	Not Evidenced	This feature may exist, but there is no clear proof that it has passed alpha testing.
Product search	Some repository structure suggests product/API support.	Not Evidenced	More test cases are needed to confirm search works correctly.
User account/login	No clear evidence was found in the reviewed material.	Not Evidenced	Login/register functionality should be tested before closed beta if it is part of the app.
Error handling	No detailed test results were found for invalid inputs, missing products, network errors, or API failures.	Not Evidenced	Closed beta users may experience problems if error handling is not tested.
Accessibility and usability	No clear accessibility testing evidence was found.	Not Evidenced	Since the app is health-related, usability and accessibility should be checked before beta.
Documentation for new	Setup and environment information exists.	Partially Pass	Documentation is useful, but setup confusion and missing clear troubleshooting steps may affect new

5. Passed Features

Some areas show positive progress and can be considered passed or close to passed.

5.1 Environment Configuration

The environment setup is clearly explained. The app uses public Expo environment variables to choose between backend mode and Firestore mode. This is useful because developers can test the app with different data sources.

Example variables include:

```
EXPO_PUBLIC_API_BASE_URL=http://127.0.0.1:8000
EXPO_PUBLIC_API_SOURCE=firestore
EXPO_PUBLIC_RECOMMENDATION_SOURCE=firestore
```

This shows that the app has a flexible configuration design. However, the team should still clearly explain when to use backend mode and when to use Firestore mode.

5.2 Food Health Evaluation Research

The research evidence supports a more balanced food recommendation model. It explains that calories are important, but calories alone are not enough to judge whether a food is healthy. Other factors such as sugar, saturated fat, protein, fibre, sodium, ingredient quality, and dietary needs should also be considered.

This is a strength of the project because it avoids giving users overly simple or misleading food advice.

6. Partially Passed Features

Several features appear to be developed or documented, but there is not enough testing evidence to mark them as fully passed.

6.1 Mobile App Setup

The mobile app setup instructions are available, including installing dependencies and starting the app with Expo. However, during setup, an npm error occurred involving the npm cache and Expo package installation. This type of issue could affect other new testers or team members.

This does not mean the app is broken, but it shows that setup still needs better troubleshooting documentation.

6.2 Backend API Connection

The backend API connection is included in the environment configuration. However, using `http://127.0.0.1:8000` can be confusing when testing on a real phone. On a physical mobile device, `127.0.0.1` refers to the phone itself, not the developer's computer. This means real-device testing may fail unless the developer uses the computer's local IP address.

This issue could block closed beta testing if not clearly explained.

6.3 Firestore Recommendation Mode

Firestore recommendation mode is available through environment settings. This is useful because the app can fetch products and run recommendation logic without relying only on the backend. However, Firestore requires correct product data and indexes. If product documents do not include the required fields, such as categories, recommendation queries may not work correctly.

Before beta testing, the team should confirm that Firestore contains enough real product data and that required indexes are created.

6.4 Recommendation Accuracy

The Food Remedy recommendation feature is one of the most important parts of the app. The research supports a balanced scoring model, but more evidence is needed to prove that the implemented recommendation system works correctly. For example, the team should test products with high sugar, high sodium, high protein, low fibre, allergy risks, and different dietary needs.

At this stage, recommendation logic should be considered partially passed, not fully passed.

7. Failed or Not Evidenced Features

Some features could not be confirmed from the available evidence.

7.1 Barcode Scanning

Barcode scanning is likely to be an important feature for a food product app. However, I tried to scan an item that not from local grocery and it couldn't show up the information, so the app is not globally yet.

7.2 Recommendation personalisation

The biggest issue is with the recommendation/Compare feature.

In product.tsx, the Compare tab is currently commented out:

```
// import RecommendationsTab from "../ProductTabs/RecommendationsTab";
```

and the tab list only has:

```
const tabs: TabKey[] = ["Nutrients", "Ingredients", "For you"];
```

So even though a RecommendationsTab.tsx file exists, the current product screen does not appear to show the Compare tab.

This matches the existing TEST010 report, which says:

Dietary preferences Vegan / Vegetarian are not considered properly.

Nutrition Guardrail Level has no observable impact.

Alternative recommendations are generic and not very relevant.

8. Closed Beta Readiness Assessment

Based on the reviewed evidence, Food Remedy is **not fully ready for closed beta testing yet**.

The project has useful documentation and some important technical foundations. The use of React Native and Expo supports cross-platform mobile development. The environment variables also allow the app to switch between backend and Firestore data sources. The food-health scoring research gives the project a reasonable direction for making better recommendations.

However, closed beta testing requires stronger evidence that the product is stable enough for selected external users. At this stage, several important features are only partially tested or not evidenced. These include backend connection, Firestore data readiness, barcode scanning, recommendation accuracy, account/profile functions, real-device testing, and error handling.

The main problem is not only whether the features exist. The main problem is that there is not enough recorded evidence to prove that the features have passed alpha testing. For a

capstone project, evidence is important because it shows that the team has tested the product properly and can justify moving to the next stage.

9. Recommendation

The recommendation is:

Food Remedy should not move into closed beta testing yet. Alpha testing should be considered partially complete, but more testing evidence and issue resolution are required before closed beta begins.

Before closed beta, the team should complete the following actions:

1. Create a formal alpha testing checklist for all main features.
2. Test the app setup process on a clean machine.
3. Confirm the app runs successfully through Expo on real iOS and Android devices.
4. Fix or document npm/Expo setup errors.
5. Confirm backend API connection using the correct local network IP.
6. Confirm Firestore product data structure and required indexes.
7. Test barcode scanning with valid, invalid, and unknown barcodes.
8. Test recommendation accuracy using different food examples.
9. Test error handling for no internet, missing product, failed API, and invalid data.
10. Record evidence for each test, including screenshots, test results, dates, and tester names.

10. Conclusion

This audit found that Food Remedy has strong evidence for several backend and data pipeline areas. Product API mapping, dataset validation, error handling, environment configuration and performance testing are well documented. The repository also contains useful automated tests and structured project documentation.

However, the application is not fully ready for closed beta because several core user-facing and personalisation features still require fixes or stronger evidence. The most important problems are dietary preference handling, nutrition guardrail behaviour and recommendation relevance. These issues directly affect the app's main purpose as a personalised food recommendation tool.

The final recommendation is that Food Remedy should remain in alpha or internal pilot stage until these issues are resolved and regression testing is completed. Once the team provides missing evidence for authentication, shopping cart, mobile compatibility, CI/CD and security testing, the application can be reassessed for closed beta readiness.

11. References

GitHub. (2026). Food Remedy API repository.

GitHub. (2026). Food Remedy API Research 2026 T1 folder.

GitHub Docs. (2026). About issues.

GitHub Docs. (2026). About pull requests.

GitHub Docs. (2026). GitHub Actions documentation.

GitHub Docs. (2026). Status checks.

IBM. (2025). What is software testing?

International Software Testing Qualifications Board. (2026). Acceptance testing.

OWASP Foundation. (2026). Web Security Testing Guide.